

Appendix A. GUI Programming

Introduction to GUI Programming

The purpose of this appendix is not to teach you how to do Graphical User Interfaces. It is simply meant to show how writing graphical applications is the same as writing other applications, just using an additional library to handle the graphical parts. As a programmer you need to get used to learning new libraries. Most of your time will be spent passing data from one library to another.

The GNOME Libraries

The GNOME project is one of several projects to provide a complete desktop to Linux users. The GNOME project includes a panel to hold application launchers and mini-applications called applets, several standard applications to do things such as file management, session management, and configuration, and an API for creating applications which fit in with the way the rest of the system works.

One thing to notice about the GNOME libraries is that they constantly create and give you pointers to large data structures, but you never need to know how they are laid out in memory. All manipulation of the GUI data structures are done entirely through function calls. This is a characteristic of good library design. Libraries change from version to version, and so does the data that each data structure holds. If you had to access and manipulate that data yourself, then when the library is updated you would have to modify your programs to work with the new library, or at least recompile them. When you access the data through functions, the functions take care of knowing where in the structure each piece of data is. The pointers you receive from the library are *opaque* - you don't need to know specifically what the structure they are pointing to looks like, you only need to know the functions that will properly manipulate it. When designing libraries, even for use within only one program, this is a good practice to keep in mind.

Appendix A. GUI Programming

This chapter will not go into details about how GNOME works. If you would like to know more, visit the GNOME developer web site at <http://developer.gnome.org/>. This site contains tutorials, mailing lists, API documentation, and everything else you need to start programming in the GNOME environment.

A Simple GNOME Program in Several Languages

This program will simply show a Window that has a button to quit the application. When that button is clicked it will ask you if you are sure, and if you click yes it will close the application. To run this program, type in the following as `gnome-example.s`:

```
#PURPOSE:  This program is meant to be an example
#           of what GUI programs look like written
#           with the GNOME libraries
#
#INPUT:     The user can only click on the "Quit"
#           button or close the window
#
#OUTPUT:    The application will close
#
#PROCESS:   If the user clicks on the "Quit" button,
#           the program will display a dialog asking
#           if they are sure.  If they click Yes, it
#           will close the application.  Otherwise
#           it will continue running
#

.section .data

###GNOME definitions - These were found in the GNOME
#                      header files for the C language
#                      and converted into their assembly
```

```
#                                equivalents

#GNOME Button Names
GNOME_STOCK_BUTTON_YES:
.ascii "Button_Yes\0"
GNOME_STOCK_BUTTON_NO:
.ascii "Button_No\0"

#Gnome MessageBox Types
GNOME_MESSAGE_BOX_QUESTION:
.ascii "question\0"

#Standard definition of NULL
.equ NULL, 0

#GNOME signal definitions
signal_destroy:
.ascii "destroy\0"
signal_delete_event:
.ascii "delete_event\0"
signal_clicked:
.ascii "clicked\0"

###Application-specific definitions

#Application information
app_id:
.ascii "gnome-example\0"
app_version:
.ascii "1.000\0"
app_title:
.ascii "Gnome Example Program\0"

#Text for Buttons and windows
button_quit_text:
```

Appendix A. GUI Programming

```
.ascii "I Want to Quit the GNOME Example Program\0"  
quit_question:  
.ascii "Are you sure you want to quit?\0"
```

```
.section .bss
```

```
#Variables to save the created widgets in  
.equ WORD_SIZE, 4  
.lcomm appPtr, WORD_SIZE  
.lcomm btnQuit, WORD_SIZE
```

```
.section .text
```

```
.globl main  
.type main,@function  
main:
```

```
    pushl %ebp  
    movl  %esp, %ebp
```

```
    #Initialize GNOME libraries
```

```
    pushl 12(%ebp)    #argv  
    pushl 8(%ebp)     #argc  
    pushl $app_version  
    pushl $app_id  
    call  gnome_init  
    addl  $16, %esp    #recover the stack
```

```
    #Create new application window
```

```
    pushl $app_title    #Window title  
    pushl $app_id       #Application ID  
    call  gnome_app_new  
    addl  $8, %esp       #recover the stack  
    movl  %eax, appPtr   #save the window pointer
```

```
#Create new button
pushl $button_quit_text #button text
call  gtk_button_new_with_label
addl  $4, %esp           #recover the stack
movl  %eax, btnQuit      #save the button pointer

#Make the button show up inside the application window
pushl btnQuit
pushl appPtr
call  gnome_app_set_contents
addl  $8, %esp

#Makes the button show up (only after it's window
#shows up, though)
pushl btnQuit
call  gtk_widget_show
addl  $4, %esp

#Makes the application window show up
pushl appPtr
call  gtk_widget_show
addl  $4, %esp

#Have GNOME call our delete_handler function
#whenever a "delete" event occurs
pushl $NULL              #extra data to pass to our
                        #function (we don't use any)
pushl $delete_handler    #function address to call
pushl $signal_delete_event #name of the signal
pushl appPtr             #widget to listen for events on
call  gtk_signal_connect
addl  $16, %esp          #recover stack

#Have GNOME call our destroy_handler function
#whenever a "destroy" event occurs
```

Appendix A. GUI Programming

```
pushl $NULL          #extra data to pass to our
                      #function (we don't use any)
pushl $destroy_handler #function address to call
pushl $signal_destroy #name of the signal
pushl appPtr          #widget to listen for events on
call  gtk_signal_connect
addl  $16, %esp        #recover stack
```

```
#Have GNOME call our click_handler function
#whenever a "click" event occurs. Note that
#the previous signals were listening on the
#application window, while this one is only
#listening on the button
```

```
pushl $NULL
pushl $click_handler
pushl $signal_clicked
pushl btnQuit
call  gtk_signal_connect
addl  $16, %esp
```

```
#Transfer control to GNOME. Everything that
#happens from here out is in reaction to user
#events, which call signal handlers. This main
#function just sets up the main window and connects
#signal handlers, and the signal handlers take
#care of the rest
call  gtk_main
```

```
#After the program is finished, leave
movl  $0, %eax
leave
ret
```

```
#A "destroy" event happens when the widget is being
#removed. In this case, when the application window
```

```
#is being removed, we simply want the event loop to
#quit
destroy_handler:
    pushl %ebp
    movl  %esp, %ebp

    #This causes gtk to exit it's event loop
    #as soon as it can.
    call  gtk_main_quit

    movl  $0, %eax
    leave
    ret

#A "delete" event happens when the application window
#gets clicked in the "x" that you normally use to
#close a window
delete_handler:
    movl  $1, %eax
    ret

#A "click" event happens when the widget gets clicked
click_handler:
    pushl %ebp
    movl  %esp, %ebp

    #Create the "Are you sure" dialog
    pushl $NULL                    #End of buttons
    pushl $GNOME_STOCK_BUTTON_NO  #Button 1
    pushl $GNOME_STOCK_BUTTON_YES #Button 0
    pushl $GNOME_MESSAGE_BOX_QUESTION #Dialog type
    pushl $quit_question          #Dialog mesasge
    call  gnome_message_box_new
    addl  $16, %esp                #recover stack
```

Appendix A. GUI Programming

`%%eax` now holds the pointer to the dialog window

#Setting Modal to 1 prevents any other user
#interaction while the dialog is being shown

```
pushl $1
pushl %eax
call  gtk_window_set_modal
popl  %eax
addl  $4, %esp
```

#Now we show the dialog

```
pushl %eax
call  gtk_widget_show
popl  %eax
```

#This sets up all the necessary signal handlers
#in order to just show the dialog, close it when
#one of the buttons is clicked, and return the
#number of the button that the user clicked on.

#The button number is based on the order the buttons
#were pushed on in the `gnome_message_box_new` function

```
pushl %eax
call  gnome_dialog_run_and_close
addl  $4, %esp
```

#Button 0 is the Yes button. If this is the
#button they clicked on, tell GNOME to quit
#it's event loop. Otherwise, do nothing

```
cmpl  $0, %eax
jne   click_handler_end
```

```
call  gtk_main_quit
```

`click_handler_end:`

```
leave
```

```
ret
```

To build this application, execute the following commands:

```
as gnome-example.s -o gnome-example.o
gcc gnome-example.o `gnome-config --libs gnomeui` \
    -o gnome-example
```

Then type in `./gnome-example` to run it.

This program, like most GUI programs, makes heavy use of passing pointers to functions as parameters. In this program you create widgets with the GNOME functions and then you set up functions to be called when certain events happen. These functions are called *callback* functions. All of the event processing is handled by the function `gtk_main`, so you don't have to worry about how the events are being processed. All you have to do is have callbacks set up to wait for them.

Here is a short description of all of the GNOME functions that were used in this program:

`gnome_init`

Takes the command-line arguments, argument count, application id, and application version and initializes the GNOME libraries.

`gnome_app_new`

Creates a new application window, and returns a pointer to it. Takes the application id and the window title as arguments.

`gtk_button_new_with_label`

Creates a new button and returns a pointer to it. Takes one argument - the text that is in the button.

Appendix A. GUI Programming

`gnome_app_set_contents`

This takes a pointer to the gnome application window and whatever widget you want (a button in this case) and makes the widget be the contents of the application window

`gtk_widget_show`

This must be called on every widget created (application window, buttons, text entry boxes, etc) in order for them to be visible. However, in order for a given widget to be visible, all of it's parents must be visible as well.

`gtk_signal_connect`

This is the function that connects widgets and their signal handling callback functions. This function takes the widget pointer, the name of the signal, the callback function, and an extra data pointer. After this function is called, any time the given event is triggered, the callback will be called with the widget that produced the signal and the extra data pointer. In this application, we don't use the extra data pointer, so we just set it to NULL, which is 0.

`gtk_main`

This function causes GNOME to enter into it's main loop. To make application programming easier, GNOME handles the main loop of the program for us. GNOME will check for events and call the appropriate callback functions when they occur. This function will continue to process events until `gtk_main_quit` is called by a signal handler.

`gtk_main_quit`

This function causes GNOME to exit it's main loop at the earliest opportunity.

`gnome_message_box_new`

This function creates a dialog window containing a question and response

buttons. It takes as parameters the message to display, the type of message it is (warning, question, etc), and a list of buttons to display. The final parameter should be NULL to indicate that there are no more buttons to display.

`gtk_window_set_modal`

This function makes the given window a modal window. In GUI programming, a modal window is one that prevents event processing in other windows until that window is closed. This is often used with Dialog windows.

`gnome_dialog_run_and_close`

This function takes a dialog pointer (the pointer returned by `gnome_message_box_new` can be used here) and will set up all of the appropriate signal handlers so that it will run until a button is pressed. At that time it will close the dialog and return to you which button was pressed. The button number refers to the order in which the buttons were set up in `gnome_message_box_new`.

The following is the same program written in the C language. Type it in as `gnome-example-c.c`:

```
/* PURPOSE:  This program is meant to be an example
              of what GUI programs look like written
              with the GNOME libraries
*/

#include <gnome.h>

/* Program definitions */
#define MY_APP_TITLE "Gnome Example Program"
#define MY_APP_ID "gnome-example"
#define MY_APP_VERSION "1.000"
```

Appendix A. GUI Programming

```
#define MY_BUTTON_TEXT "I Want to Quit the Example Program"
#define MY_QUIT_QUESTION "Are you sure you want to quit?"

/* Must declare functions before they are used */
int destroy_handler(gpointer window,
    GdkEventAny *e,
    gpointer data);
int delete_handler(gpointer window,
    GdkEventAny *e,
    gpointer data);
int click_handler(gpointer window,
    GdkEventAny *e,
    gpointer data);

int main(int argc, char **argv)
{
    gpointer appPtr; /* application window */
    gpointer btnQuit; /* quit button */

    /* Initialize GNOME libraries */
    gnome_init(MY_APP_ID, MY_APP_VERSION, argc, argv);

    /* Create new application window */
    appPtr = gnome_app_new(MY_APP_ID, MY_APP_TITLE);

    /* Create new button */
    btnQuit = gtk_button_new_with_label(MY_BUTTON_TEXT);

    /* Make the button show up inside the application window */
    gnome_app_set_contents(appPtr, btnQuit);

    /* Makes the button show up */
    gtk_widget_show(btnQuit);

    /* Makes the application window show up */
```

```
gtk_widget_show(appPtr);

/* Connect the signal handlers */
gtk_signal_connect(appPtr, "delete_event",
    GTK_SIGNAL_FUNC(delete_handler), NULL);
gtk_signal_connect(appPtr, "destroy",
    GTK_SIGNAL_FUNC(destroy_handler), NULL);
gtk_signal_connect(btnQuit, "clicked",
    GTK_SIGNAL_FUNC(click_handler), NULL);

/* Transfer control to GNOME */
gtk_main();

return 0;
}

/* Function to receive the "destroy" signal */
int destroy_handler(gpointer window,
    GdkEventAny *e,
    gpointer data)
{
    /* Leave GNOME event loop */
    gtk_main_quit();
    return 0;
}

/* Function to receive the "delete_event" signal */
int delete_handler(gpointer window,
    GdkEventAny *e,
    gpointer data)
{
    return 0;
}
```

Appendix A. GUI Programming

```
/* Function to receive the "clicked" signal */
int click_handler(gpointer window,
    GdkEventAny *e,
    gpointer data)
{
    gpointer msgbox;
    int buttonClicked;

    /* Create the "Are you sure" dialog */
    msgbox = gnome_message_box_new(
        MY_QUIT_QUESTION,
        GNOME_MESSAGE_BOX_QUESTION,
        GNOME_STOCK_BUTTON_YES,
        GNOME_STOCK_BUTTON_NO,
        NULL);
    gtk_window_set_modal(msgbox, 1);
    gtk_widget_show(msgbox);

    /* Run dialog box */
    buttonClicked = gnome_dialog_run_and_close(msgbox);

    /* Button 0 is the Yes button. If this is the
    button they clicked on, tell GNOME to quit
    it's event loop. Otherwise, do nothing */
    if(buttonClicked == 0)
    {
        gtk_main_quit();
    }

    return 0;
}
```

To compile it, type

```
gcc gnome-example-c.c 'gnome-config --cflags \  
--libs gnomeui' -o gnome-example-c
```

Run it by typing `./gnome-example-c`.

Finally, we have a version in Python. Type it in as `gnome-example.py`:

```
#PURPOSE:  This program is meant to be an example  
#          of what GUI programs look like written  
#          with the GNOME libraries  
#  
  
#Import GNOME libraries  
import gtk  
import gnome.ui  
  
#####DEFINE CALLBACK FUNCTIONS FIRST#####  
  
#In Python, functions have to be defined before  
#they are used, so we have to define our callback  
#functions first.  
  
def destroy_handler(event):  
    gtk.mainquit()  
    return 0  
  
def delete_handler(window, event):  
    return 0  
  
def click_handler(event):  
    #Create the "Are you sure" dialog  
    msgbox = gnome.ui.GnomeMessageBox(  
        "Are you sure you want to quit?",  
        gnome.ui.MESSAGE_BOX_QUESTION,  
        gnome.ui.STOCK_BUTTON_YES,  
        gnome.ui.STOCK_BUTTON_NO)
```

Appendix A. GUI Programming

```
msgbox.set_modal(1)
msgbox.show()

result = msgbox.run_and_close()

#Button 0 is the Yes button. If this is the
#button they clicked on, tell GNOME to quit
#it's event loop. Otherwise, do nothing
if (result == 0):
    gtk.mainquit()

return 0

####MAIN PROGRAM####

#Create new application window
myapp = gnome.ui.GnomeApp(
    "gnome-example", "Gnome Example Program")

#Create new button
mybutton = gtk.GtkButton(
    "I Want to Quit the GNOME Example program")
myapp.set_contents(mybutton)

#Makes the button show up
mybutton.show()

#Makes the application window show up
myapp.show()

#Connect signal handlers
myapp.connect("delete_event", delete_handler)
myapp.connect("destroy", destroy_handler)
mybutton.connect("clicked", click_handler)
```

```
#Transfer control to GNOME  
gtk.mainloop()
```

To run it type `python gnome-example.py`.

GUI Builders

In the previous example, you have created the user-interface for the application by calling the create functions for each widget and placing it where you wanted it. However, this can be quite burdensome for more complex applications. Many programming environments, including GNOME, have programs called GUI builders that can be used to automatically create your GUI for you. You just have to write the code for the signal handlers and for initializing your program. The main GUI builder for GNOME applications is called GLADE. GLADE ships with most Linux distributions.

There are GUI builders for most programming environments. Borland has a range of tools that will build GUIs quickly and easily on Linux and Win32 systems. The KDE environment has a tool called QT Designer which helps you automatically develop the GUI for their system.

There is a broad range of choices for developing graphical applications, but hopefully this appendix gave you a taste of what GUI programming is like.

